

Operational Evals — Interview Notes (Concept-Clear)

Latency, Cost aur Reliability — RAG Application ko Production-Ready Banane Ka Framework

1. Kahan Hum Abhi Hain? (Recap)

RAG Eval Suite **3 layers** mein bani: **Component** (retriever + generator) — **[Done], Pipeline** (RAG Triad) — **[Done]**, aur **Application** (Quality, Safety, Ops). Application Quality (Correctness/Completeness/Style) aur Safety (Toxicity/Leakage/Scope) already cover ho chuke hain. **Aaj: Operations**. Regression Testing agla session hai (isi session mein plan tha lekin time constraint ki wajah se split ho gaya).

Is session ke baad poora **offline RAG Eval Suite complete** ho jaayega — jo agle session mein Regression Testing ke liye use hoga.

2. Operational Evals Kya Hote Hain?

Operational Evals ek **different question** answer karte hain quality evals (Correctness, Faithfulness, Relevance) se. Ab tak jitne bhi evals padhe — component, pipeline, ya application level — sab ye batate the ki **application ka output kitna accha hai** (quality).

"Even if the RAG system gives good answers, can it run reliably, quickly, and economically in production?" — yehi sawaal Operational Evals answer karte hain.

Quality Evals vs Operational Evals

	Quality Evals	Operational Evals
Kya Check Karte Hain	Output ki quality (correctness, faithfulness, relevance)	System reliably, quickly, cheaply chal raha hai ya nahi
LLM-as-Judge Chahiye?	Haan, zyadatar mein	Nahi — koi LLM judge involved nahi hota
Golden Dataset Chahiye?	Kai jagah haan	Nahi — sirf software/telemetry-driven measurements
Nature	Judgment-based	Simple software measurements (jaise counters, timers)

3. 3 Main Operational Evals

Metric	Kya Batata Hai
Latency	Users ko answer dekhne ke liye kitna wait karna padta hai
Cost	Per query answer karne mein kitna paisa kharch ho raha hai
Reliability	Pipeline kitni baar successfully chala aur kitni baar error aaya

Ek chautha bhi hota hai — **Throughput** (ek given time mein kitne parallel requests handle kar sakte ho) — lekin isko test karne ke liye **load testing** chahiye hoti hai jo is session ke scope se bahar hai.

4. Kya Operational Evals Offline Suite Mein Hone Chahiye?

Logically lagta hai ki Operational Evals sirf **post-deployment** useful hain — kyunki definition hi ye kehti hai ki "production mein system reliably chal raha hai ya nahi". To sawaal uthta hai: kya deployment se pehle offline suite mein ye rakhna time waste karna hai?

Answer: Nahi, offline suite mein bhi zaroori hai. Reason ek scenario se samajhte hain:

Scenario — Ek Live Example

Maan lo aapne RAG application banaya: top-5 chunks retriever se le rahe ho, LLM ko de rahe ho. Quality metrics achhe aa rahe hain — Correctness 91%, Faithfulness 94%, Answer Relevance 93%. Operational metrics bhi measure kiye:

Metric	Before Changes	After Changes*
Correctness	91%	95%
Faithfulness	94%	96%
Answer Relevance	93%	95%
Avg Latency	2.3s	4.1s
P95 Latency	4.8s	6.2s
Avg Cost/Query	₹0.72	₹1.08
Timeout Rate	2%	1% (improved)

*Changes: reranker add kiya, top-5 se top-10 chunks kar diye, chhote model ko bade model se replace kiya.

Quality improve hui hai, lekin **latency aur cost bhi kaafi badh gaye**. Agar operational evals measure hi nahi karte, to ye poora trade-off miss ho jaata — aur deployment ke baad users complain karte ki "app slow ho gaya" aur bill achanak badh jaata bina pata chale.

Key Insight

Operational eval ki **absolute value** laptop pe utni reliable nahi hoti (production mein number alag ho sakta hai). Lekin jo **differential/direction** hai — ki koi change karne se latency/cost badha ya ghata — **wo trustworthy hai** aur bahut important signal deta hai. Isi wajah se offline suite mein operational evals rakhna chahiye.

"Do not wait until production to discover that your RAG pipeline is too slow and too expensive."

Assumption: Jab bhi do experiments compare karo, setup bilkul same rakho — warna direction bhi reliable nahi rahegi.

5. Latency — Deep Dive

Definition: Latency = amount of time a system takes to respond to a request. User ne question type kiya, enter dabaya — kitni der mein poora reply screen pe dikha, wahi latency hai.

Concept: Basic Idea (Timer/Counter)

Conceptually simple: request start hone pe ek timer start karo, response aane pe stop karo, difference nikaal lo. Lekin practically ye itna straightforward nahi hai — **7 important considerations** hain jo dhyan mein rakhne chahiye.

Consideration 1 — Distributions Over Averages (P50, P95, P99)

Sirf **mean latency** mat batao. Production mein latency ek time window (last 1 hour, last 24 hours) mein measure hoti hai — us window mein har request ki apni latency value hoti hai, jo milke ek **distribution** banati hai.

Percentile	Matlab
P50 (Median)	50% requests is time ya usse kam mein complete hue
P95	95% requests is time ya usse kam mein complete hue
P99	99% requests is time ya usse kam mein complete hue

Mean sirf average dikhata hai; P95/P99 wo **"tail latency"** dikhate hain jo un users ka experience batati hai jinko sabse zyada wait karna pada. Wahi worst-case scenario samajhne ke liye zaroori hai.

Consideration 2 — End-to-End Ke Sath Component-Level Breakdown Bhi Do

Sirf ye mat batao ki total 4.5 seconds lage. Ye bhi batao ki usme se kitna time **Retriever** ne liya aur kitna **Generator** ne liya. Isse pata chalta hai ki improvement ka scope kahan hai — embedding, retrieval, reranking, generation — har stage ka time alag-alag dikhao.

Consideration 3 — TTFT (Time To First Token) Alag Se Measure Karo

LLM apps **stream** karte hain — jaise hi pehla token milta hai, wo turant screen pe dikhana shuru ho jaata hai (isiliye ChatGPT/Claude ka jawab "type ho raha" jaisa dikhta hai). Iske 2 fayde hain: (1) bada text ek saath dikhne se padhne mein dikkat hoti hai, (2) agar poora answer generate hone tak wait kiya jaaye to user ko lagta hai kuch ho hi nahi raha — streaming se turant movement dikhti hai.

TTFT = pehla token screen pe dikhne mein kitna time laga. Ye end-to-end latency se alag measure karna chahiye kyunki ye user experience ka bahut important factor hai.

Consideration 4 — Cold Starts Ka Dhyan Rakho

Cold Start = jab connection initialize ho raha ho, model load ho raha ho, vector DB se connection setup ho raha ho, network handshake ho raha ho, cache initialize ho raha ho, ya container/serverless code start ho raha ho — in sab cases mein pehle request ke liye extra time lagta hai.

Isliye jab latency measure karo, **pehle 1-2 questions ko skip karo** (warm-up runs) — 3rd question se measurement start karo, taaki cold-start ka effect data mein na aaye.

Consideration 5 — Token Count aur Context Size Bhi Report Karo

Latency directly depend karti hai answer ki length aur context size par — bada answer/context zyada time lega, chhota kam. Isliye latency value ke saath token count/context size bhi batana chahiye, taaki numbers ka context samajh aaye.

Consideration 6 — Latency Ko Throughput Se Distinguish Karo

	Latency	Throughput
Kya Batata Hai	Ek single request ko process karne mein kitna time laga	Ek given time mein kitne parallel requests handle ho sakte hain
Restaurant Analogy	Ek customer ko table pe khana aane mein kitna wait karna pada	Kitchen ek ghante mein kitne customers serve kar sakti hai

Jab tak system apne throughput threshold ke andar operate karta hai, latency theek rehti hai. Threshold cross hote hi queue banti hai aur baad wale requests ki latency badh jaati hai. Isliye dono ko saath discuss karna chahiye.

Consideration 7 — Multiple Runs Repeat Karo (External APIs Noisy Hoti Hain)

External LLM APIs pe kabhi-kabhi server-side noise/variance hota hai. Isliye ek hi question ko **ek baar nahi, multiple baar (jaise 5 baar)** bhejo aur average nikaalo — taaki ek reliable number mile, sirf ek fluke value nahi.

Consideration 8 — Failures Ko Latency Se Alag Track Karo

Agar latency achhi dikhe (jaise P95 = 2s) lekin timeout rate bhi badh gaya (jaise 8%) — to ye **false sense of improvement** deta hai. Reality ye ho sakti hai ki bahut saare requests answer hi nahi ho paaye, aur jo answer hue unki latency kam thi, isliye overall number achha dikh raha hai. Isliye latency ke saath timeout rate bhi hamesha dekho.

Consideration 9 — Latency Budgets Define Karo

Apne business/application ke hisaab se ek **threshold (SLO — Service Level Objective)** decide karo — system level pe bhi, component level pe bhi. Jaise: "P95 end-to-end latency 3 seconds se zyada nahi honi chahiye" ya "Retriever ko documents laane mein 1 second se zyada nahi lagna chahiye". Ye number apni application, users, aur industry standards se decide hota hai.

Consideration 10 — Representative aur Segmented Workloads Use Karo

Test dataset mein sirf easy questions mat rakho. Simple, medium, aur complex — **har type ke questions** ka representation hona chahiye, taaki real-world jaisi latency value mile. Warna sirf simple questions se latency kam dikhegi, lekin production mein log har tarah ke complex questions poochenge.

Class Ka Live Result — Latency

Setup: 5 questions × 5 repeats = 25 calls, 2 warm-up runs discard kiye, SLO budget: End-to-end P95 ≤ 3000ms, TTFT P95 ≤ 1200ms.

3.6s Mean End-to-End Latency	5.3s P95 / P99 Latency
---------------------------------	---------------------------

Breakdown	Time
Retrieval	~700-760 ms
Generation	~2.9 s (almost 4x zyada retrieval se)
Mean TTFT	1.6 s
P95 TTFT	2.0 s

Result: Dono SLOs FAIL ho rahe the — End-to-End P95 (5.3s) budget (3s) se bahut zyada, aur TTFT P95 (2081ms) bhi budget (1200ms) se zyada. Iska matlab: pipeline mein improvements chahiye.

Latency Reduce Karne Ke Tareeke

Area	Kya Kar Sakte Ho
Generator Time Ghatana	Faster/lighter model variant use karo, model router lagao (simple question → chhota model, complex → bada model), concise answers ka prompt do, answer length pe hard cap lagao
Context Size Ghatana	Retrieval ka "k" value kam karo (jaise 10 se 5), contextual compression use karo
Retriever Optimize Karna	Embedding, vector DB retrieval, reranking — teeno stages ko alag-alag analyse karo, jahan zyada time lag raha hai wahan improve karo
Caching	Embeddings, retrieval results, reranking results, aur system prompt — sabko cache kiya ja sakta hai agar repeat ho rahe hain
Infrastructure Distance	Vector DB, reranker API, aur LLM API sab ek hi geographic region mein rakho, taaki network hop ka extra time na lage

6. Cost — Deep Dive

Definition: Cost = monetary expense to process a user query, mainly LLM tokens consumed during generation ki wajah se drive hota hai.

Application Chalane Mein Paisa Kahan Kharch Hota Hai?

Component	Detail
LLM (Sabse Bada Kharcha)	Input aur output tokens ke hisaab se rate charge hota hai — har provider ka apna rate hota hai per model
Vector Database	Agar commercial/paid database use ho raha hai
Reranker API	Agar paid reranking service use ho rahi hai
Embedding Model	Chhota kharcha hota hai, lekin phir bhi ek factor hai
Infrastructure	Application host karne ka kharcha

Focus: LLM tokens sabse dominant factor hote hain kisi bhi LLM-based application mein — isiliye humara discussion mainly usi pe center hai (assumption: vector DB free/local hai, reranker chhota model hai, infra abhi deploy nahi hui).

Formula (concept level): Cost = Tokens Consumed × Rate (per token/per million tokens). Rate provider ke hisaab se milta hai (jaise OpenRouter ya provider ki pricing page pe). Generally, **output tokens input se ~4x zyada costly** hote hain.

Cost Measure Karne Ke Considerations

- Cost Per Query** sabse important number hai (na ki sirf total daily/monthly spend)
- Overall cost ko **input vs output** mein separately break down karo — pata chalega kahan improvement ka scope hai
- Cost ko bhi **distribution** ki tarah measure karo (P95 cost) — kuch queries "long-tail" ho sakti hain jo bahut zyada costly hoti hain
- Query type ke hisaab se segment** karo — simple/medium/complex questions mein alag-alag kitna cost aata hai
- Cost budget** set karo — business team decide karti hai monthly spend limit, use per-query limit mein translate karo

Class Ka Live Result — Cost

Setup: 4 questions × 3 repeats = 12 calls, GPT-4o-mini model.

~₹0.02 Avg Cost Per Query	1700→209 Avg Input → Output Tokens
------------------------------	---------------------------------------

Observations: Cost ka range bahut **tight/stable** tha (latency jaisa fluctuation nahi tha). Is application mein **input tokens pe zyada kharcha** ho raha tha (unlike coding agents jahan output pe zyada kharcha hota hai, kyunki wahan input chhota instruction hota hai aur output mein poora kaam hota hai). Bahut aggressive automatic prompt caching bhi dikhi (provider khud detect kar leta hai repeat system prompt/context ko).

Projections: Agar per din 2000 questions aayein — daily cost aur monthly cost calculate kiya ja sakta hai (linear scaling). Current setup **budget ke andar pass** ho raha tha.

Cost Reduce Karne Ke Tareeke

Tareeka	Detail
Context Size Ghatana	Chhote chunks ya contextual compression use karo — input tokens kam honge, output bhi shayad kam ho
System Prompt Efficient Banana	Careful editing se prompt ki length kam karo bina essence khoye

Answer Length Limit	Model ko concise answers ka instruction do
Cheaper Model	Ek sasta alternative model use karo
Caching	Prompt caching jahan bhi possible ho, input+output dono mein bacha sakti hai

Cost ko reduce karne ke tareeke Latency ki tulna mein **kam hote hain** — kyunki zyada kharcha directly LLM se aata hai. Sabse bada lever **model choice** hi hai; baaki optimizations sirf 5% jaisa chhota differential dete hain.

7. Reliability — Deep Dive

"Reliability is the ability of a RAG system to successfully serve requests without errors, timeouts, crashes, or broken pipeline stages."

Example: 10 users ne question poocha, sirf 8 ko answer mila, 2 ko "Try again after some time" dikha. Reasons kuch bhi ho sakte hain — API fail hui, reranking API fail hui, vector DB context generate nahi kar paaya, server down tha. Yahan system **80% reliable** hai.

Reliability Ke Core Metrics

Metric	Matlab
Success Rate	Kitne % requests successfully serve hue (= 1 – Error Rate)
Error Rate	Kitne % requests fail hue (= 1 – Success Rate)
Timeout Rate	Kitne % requests allowed time se zyada le liya aur time-out ho gaye
Retry Rate	Kitne % requests ko kam se kam ek retry chahiye padi

Considerations

- Overall success/failure rate** to measure karna hi hai
- Failures ko categorize karo** — ek generic error rate ki jagah break down karo: LLM API failure, retriever failure, reranker failure, timeout error, rate-limit error, parsing/formatting error, internal exceptions — har category ka % pata hona chahiye
- Load ke under reliability alag se measure karo** — pipeline single-user offline test mein highly reliable lag sakti hai, lekin concurrency badhne pe fail hona shuru ho sakti hai
- Sufficient samples use karo** — 25-50 questions bahut kam hain; asal problems 1000+ requests pe hi dikhte hain
- Representative requests use karo** — simple, long-context, complex, long-answer-producing, aur edge-case queries — sabka mix hona chahiye, taaki proper segmentation mil sake (ho sakta hai complex queries mein error rate zyada ho, simple mein kam)

Class Ka Live Result — Reliability

Setup: 4 questions × 5 repeats = 20 requests, max retries = 2.

Result: Success Rate = 100%, Error Rate = 0%, Retry needed = 0 (kisi bhi request mein retry ki zaroorat nahi padi).

Ye result bahut **ideal** hai kyunki laptop pe sirf 20 requests ek reliable API (OpenAI) ke against bheji gayi. Real-world post-deployment mein — jab thousands of concurrent users, ya kam-reliable/sasta model use ho — tab ye numbers spike karte dikhenge. Current setup "too ideal" hai bad numbers dikhane ke liye.

8. Throughput (Chautha Operational Metric — Out of Scope)

Throughput = ek given time period mein system kitne requests handle kar sakta hai (parallel/concurrent). Ye bhi ek important operational aspect hai, lekin isko properly test karne ke liye **Load Testing** / **Stress Testing** chahiye hoti hai — dedicated software se simulate kiya jaata hai ki bahut saare requests ek saath application ko hit kar rahe hain. Ye is session ke scope se bahar hai, lekin proper company setup mein ye zaroor test hota hai.

9. Key Takeaways

- Operational Evals ek **different question** answer karte hain — quality nahi, balki system **reliably, quickly, cheaply** chal raha hai ya nahi.
- Inko na **LLM-as-judge** chahiye na hi **golden dataset** — ye sirf software/telemetry-driven measurements hain (counters, timers).
- Absolute value laptop pe utni reliable nahi hoti, lekin **differential/direction** (change se kya better/worse hua) bahut trustworthy signal hai — isliye offline suite mein bhi hona chahiye.
- Latency**: mean nahi, **P50/P95/P99 distribution** dekho; end-to-end ke saath component breakdown do; TTFT alag measure karo; cold starts skip karo; token/context size report karo; latency ko throughput se distinguish karo; multiple runs average karo; failures alag track karo; budgets (SLOs) define karo; representative workload use karo.
- Cost**: mainly **LLM tokens** se drive hota hai; output tokens generally input se ~4x costly hote hain; per-query cost sabse important number; input vs output break down karo; distribution aur query-type segmentation bhi karo; sabse bada lever **model choice** hai.
- Reliability**: Success/Error/Timeout/Retry rate measure karo; failures ko category-wise break down karo; load ke under alag se test karo; enough samples aur representative requests use karo.
- Throughput** chautha metric hai, lekin load testing chahiye — is session ke scope se bahar.
- Ab poora **offline RAG Eval Suite complete** ho chuka hai — Component + Pipeline + Application (Quality + Safety + Operations).
- Agle Sessions**: Regression Testing (poore suite ko ek flow mein combine karke, koi bhi change karne pe automatically re-check karna) aur uske baad Online Evals, phir Agent Evaluations.

"Operational Evals aapko batate hain ki quality-wise achha performing system bhi production mein slow, expensive, ya unreliable ho sakta hai. Inhe deployment ke baad discover karne ke bajaye offline suite mein hi measure karna chahiye — taaki koi bhi change karne se pehle uska latency/cost/reliability impact pata chal sake."